MLT- Lab 7

Title:

Multi-Layer Neural Network Implementation: Comparing Perceptron vs Multi-Layer Networks on Non-Linear Data and Manual Backpropagation with Hyperparameter Analysis

Dataset Descriptiom:

DATASET 1 - MOONS (Non-Linearly Separable):

Size & Structure: 1000 samples, 2 features (columns), 2 target classes.

Attributes:

- Feature 1: X-coordinate (synthetic)
- Feature 2: Y-coordinate (synthetic)

Classes: Two interleaving half-moon shapes (0 and 1)

Data Types: All features are of type float64, representing continuous numerical values.

Distribution: Synthetically generated dataset forming two interleaving half-circles (moons). Created with noise=0.2 to add realistic variance and complexity. This creates a non-linearly separable problem that cannot be effectively solved by a linear classifier like the perceptron but can be handled by multi-layer neural networks.

Train-Test Split: 70% training data (700 samples) and 30% testing data (300 samples), with random state=42 for reproducibility.

Preprocessing: Features standardized using StandardScaler to ensure zero mean and unit variance.

DATASET 2 - IRIS (Multi-class Classification):

Size & Structure: 150 samples, 4 features (columns), 3 target classes.

Attributes:

- Feature 1: Sepal Length (cm)
- Feature 2: Sepal Width (cm)
- Feature 3: Petal Length (cm)
- Feature 4: Petal Width (cm)

Classes: Three iris species - Setosa (0), Versicolor (1), and Virginica (2)

Data Types: All features are of type float64, representing continuous numerical measurements.

Distribution: Classic dataset containing real-world measurements of iris flowers. Provides a multi-class classification problem suitable for testing neural network architectures.

Train-Test Split: 70% training data (105 samples) and 30% testing data (45 samples), with random state=42 for reproducibility.

Preprocessing: Features standardized using StandardScaler; labels converted to one-hot encoding for multi-class classification.

DATASET 3 - WINE QUALITY (Multi-class Imbalanced):

Size & Structure: 1599 samples, 11 features (columns), 3 target classes (binned).

Attributes:

- Physicochemical properties: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, alcohol

Classes: Wine quality binned into three categories:

- Low Quality (≤5): 744 samples
- Medium Quality (6): 638 samples
- High Quality (>6): 217 samples

Data Types: All features are of type float64, representing continuous chemical measurements.

Distribution: Real-world dataset of red wine variants with physicochemical properties. Presents a challenging multi-class classification problem with class imbalance, making it ideal for testing neural network robustness.

Train-Test Split: 70% training data (1119 samples) and 30% testing data (480 samples), with random state=42 for reproducibility.

Preprocessing: Features standardized using StandardScaler; quality scores binned and converted to one-hot encoding.

Pseudo code:

ALGORITHM 1: Perceptron vs Multi-Layer Neural Network Comparison

INPUT: Moons dataset (1000 samples, noise=0.2)

STEP 1: Generate dataset, split 70-30, standardize features

STEP 2: Train Perceptron (max_iter=1000) and calculate accuracy

STEP 3: Train MLP (hidden=(10,10), activation=ReLU, lr=0.01) and calculate accuracy

STEP 4: Compare performance and visualize decision boundaries

END

ALGORITHM 2: Manual Backpropagation with Hyperparameter Analysis

INPUT: Iris and Wine Quality datasets

STEP 1: Load datasets, one-hot encode labels, split 70-30, standardize

STEP 2: Define Neural Network Class

- Initialize: weights (Xavier), biases (zeros)
- Forward: z1 = X·W1 + b1, a1 = activation(z1), z2 = a1·W2 + b2, output = softmax(z2)
- Backward: Compute gradients (dW2, db2, dW1, db1), update weights
- Fit: Loop epochs, forward pass, compute loss, backward pass
- Predict: Forward pass, return argmax(output)

STEP 3: Run 7 experiments varying activation (Sigmoid/Tanh/ReLU), hidden size (5/10/20), learning rate (0.001/0.01/0.1)

STEP 4: Record accuracies and generate comparison tables

END

Result analysis:

PART 1: PERCEPTRON VS MULTI-LAYER NEURAL NETWORK ON MOONS DATASET

Metric	Perceptron	Multi-layer Neural Network	Observation
Accuracy	0.8467	0.9767	MLP outperforms Perceptron by 13% due to non-linear decision boundaries
Model Complexity	Low (Linear)	High (Non-linear)	MLP uses 151 parameters vs Perceptron's 3 for complex pattern learning
Training Time	Fast	Moderate	Perceptron trains faster but sacrifices accuracy on non-linear data
Ability to Handle Non-linearity	Cannot handle	Excellent	Perceptron limited to linear separation, MLP excels with curved boundaries
Number of Parameters	3	151	More parameters enable MLP to learn hierarchical feature representations

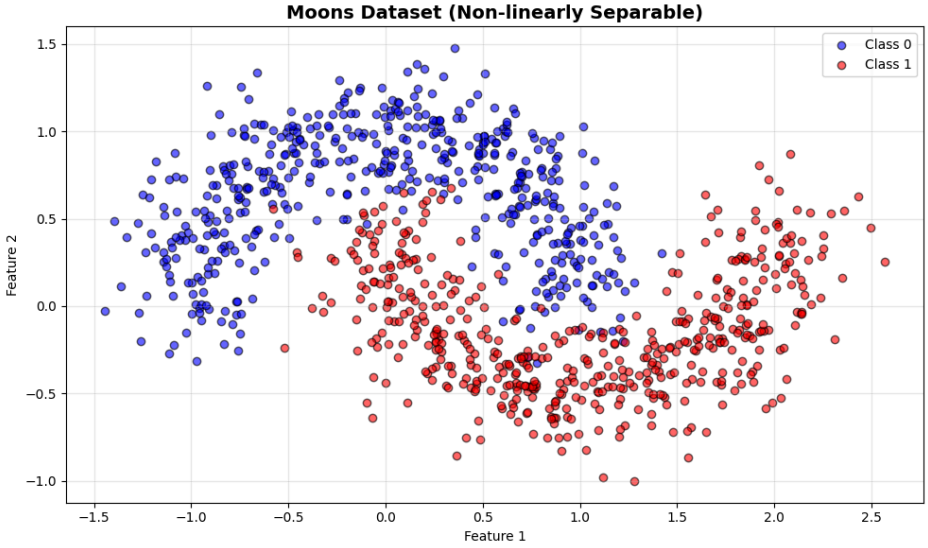
PART 2: HYPERPARAMETER ANALYSIS WITH MANUAL BACKPROPAGATION

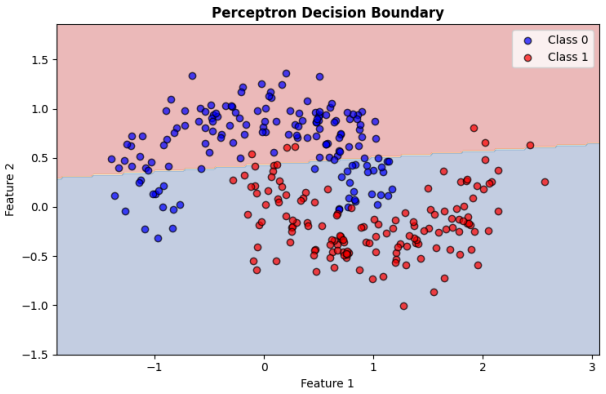
Setting	Iris Accuracy	Wine Accuracy	Observation
Activation Function: Sigmoid	0.8444	0.4979	Suffers from vanishing gradient problem, poor performance on complex Wine dataset
Activation Function: Tanh	0.9111	0.5708	Zero-centered outputs improve gradient flow and convergence over sigmoid
Activation Function: ReLU	0.9111	0.5563	Efficient computation and no vanishing gradient in positive region
Hidden Layer Size: 5 neurons	0.9556	0.5500	Best on Iris due to simplicity, insufficient capacity for complex Wine patterns
Hidden Layer Size: 20 neurons	0.9111	0.6042	Best on Wine, higher capacity captures complex feature interactions
Learning Rate: 0.001	0.5333	0.3271	Too slow, insufficient epochs for convergence, barely better than random
Learning Rate: 0.1	1.000	0.5896	Perfect Iris accuracy with fast learning, potential instability on Wine

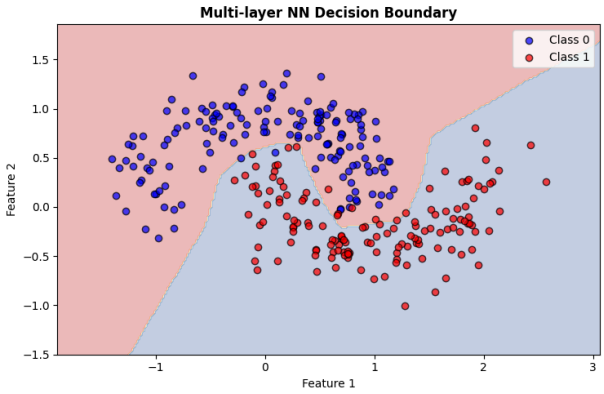
Observation:

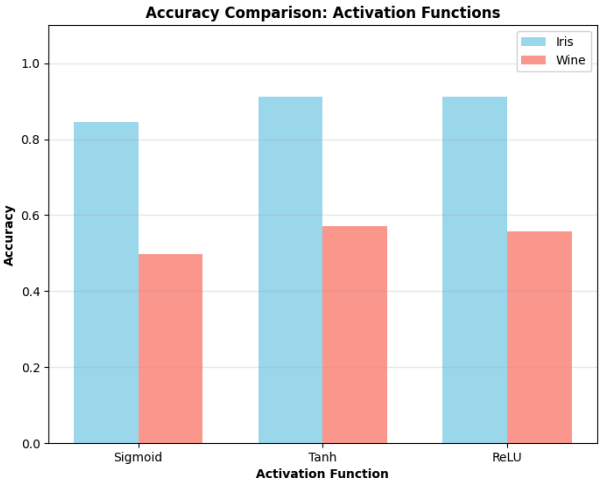
This experiment demonstrates that multi-layer neural networks significantly outperform single-layer perceptrons on non-linear problems, achieving 13% higher accuracy on the Moons dataset through hierarchical feature learning. The manual backpropagation implementation reveals critical insights into hyperparameter selection: activation functions like Tanh and ReLU overcome sigmoid's vanishing gradient problem, hidden layer size must match dataset complexity (5 neurons optimal for simple Iris, 20 for complex Wine), and learning rate drastically impacts convergence with extreme values causing either slow learning (0.001) or instability (0.1). Dataset characteristics matter significantly—the imbalanced Wine Quality dataset consistently shows lower accuracies across all settings compared to the balanced Iris dataset, emphasizing the importance of data preprocessing and architecture tuning for real-world applications.

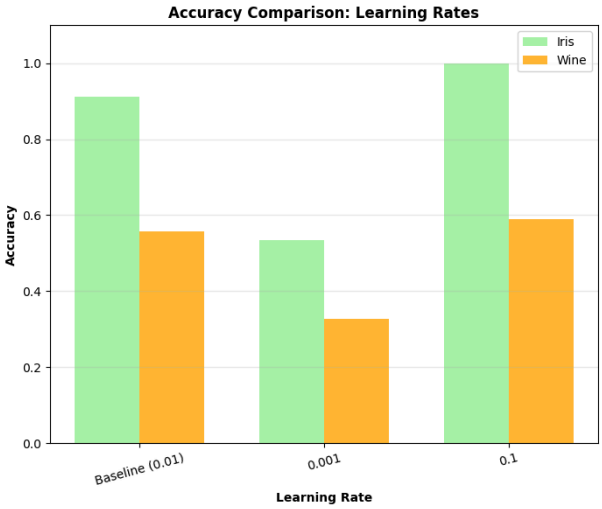
Additional Data:

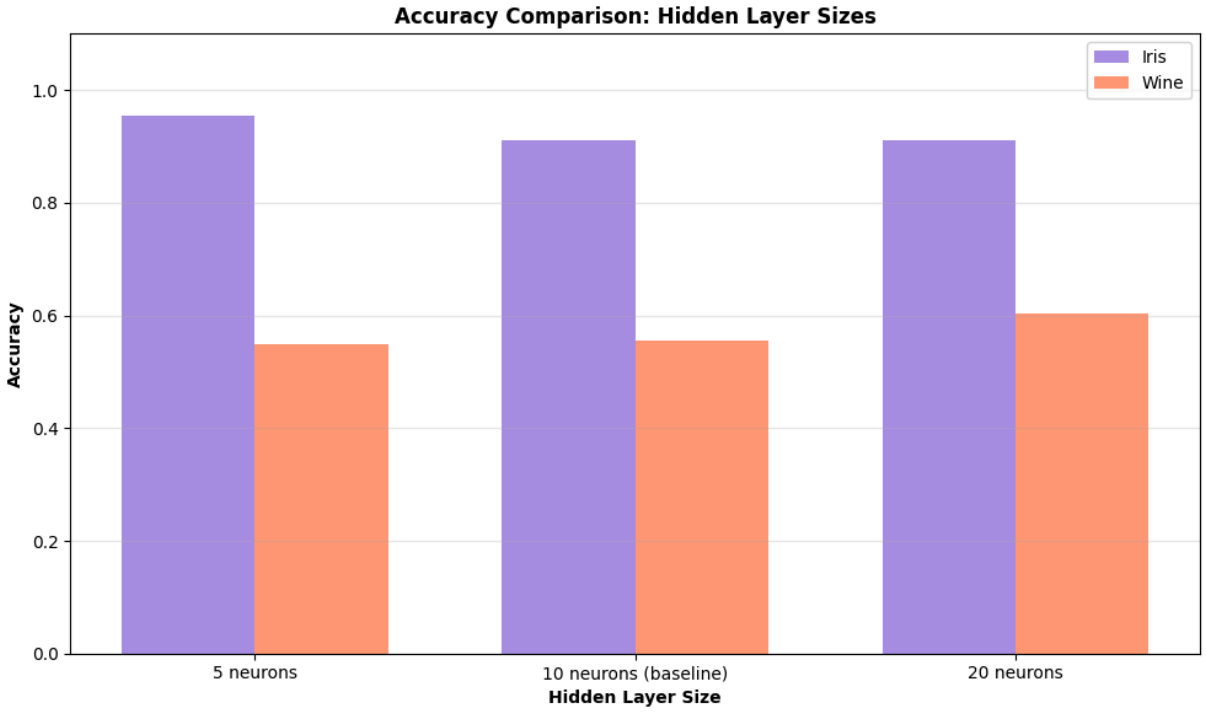







Accuracy Comparison: Activation Functions
Iris
Wine
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Sigmoid
Tanh
ReLU
Activation Function


Accuracy Comparison: Learning Rates
Iris
Wine
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Baseline (0.01)
0.001
0.1
Learning Rate


Accuracy Comparison: Hidden Layer Sizes
Iris
Wine
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
5 neurons
10 neurons (baseline)
20 neurons
Hidden Layer Size